# SMART STORAGE TANK LEVEL & USAGE PREDICTION SYSTEM

For Petrochemical Plants

## Milestone 1 — Circuit Design & Cloud Integration

**TEAM-7**

**MITHUN CHANDERAN**
**AARON ROHIT**
**RAHUL R**
**ABINAND**

Christ University Internship
Industry Partner: L&T Construction

*Sensitivity: LNT Construction Internal Use*

# 1. Project Overview

In a petrochemical plant, continuous and reliable monitoring of liquid raw material storage is critical to maintaining uninterrupted production. This project addresses the need for an intelligent, sensor-driven monitoring system capable of tracking tank levels in real time, predicting consumption trends, and providing actionable insights to plant operators.

The system consists of five cylindrical tanks, each with a capacity of 50,000 litres, which are filled daily at 6:00 AM. Liquid is supplied throughout the day based on operational demand. Since usage rates vary with production schedules, tanks may empty at different times, making manual monitoring impractical and error-prone.

This documentation covers Milestone 1 of the project, which focuses on the hardware circuit design and cloud connectivity. The circuit is built and simulated using Cirkit Designer and data is transmitted to the ThingSpeak IoT cloud platform for remote monitoring and further analysis in subsequent milestones.

## 1.1 Actual Problem

The absence of a continuous monitoring mechanism makes it extremely difficult to track real-time tank levels, measure consumption rates, and estimate time-to-empty values. This operational gap leads to critical issues including tank overflow during refilling, dry-run conditions during periods of high demand, inventory imbalances across tanks, and inefficient planning of resources and procurement.

## 1.2 Expected Solution

A smart monitoring system using ultrasonic and water level sensors with time-stamped data collection is required to continuously track tank levels and usage patterns. The system is designed to analyze consumption trends, predict critical low-level thresholds using machine learning models, and deliver real-time insights and alerts through a centralized dashboard. This improves overall safety, operational efficiency, and data-driven decision-making.

# 2. System Architecture

The end-to-end architecture for Milestone 1 follows a layered IoT pipeline. Sensors at the edge collect physical measurements and transmit the data to the ESP32-S3 microcontroller, which processes and uploads readings to the ThingSpeak cloud platform over Wi-Fi. The architecture is designed to be modular, allowing seamless extension into the machine learning and dashboard layers in subsequent milestones.

## 2.1 Architecture Layers

- Sensing Layer — HC-SR04 ultrasonic sensors measure the distance to the water surface in each tank. Water level sensors provide a complementary analog signal to cross-validate the reading.
- Processing Layer — The ESP32-S3 microcontroller reads all sensor outputs, calculates tank levels and usage metrics, and manages the Wi-Fi connection.
- Connectivity Layer — Data is transmitted over Wi-Fi using HTTP POST requests to the ThingSpeak REST API every 15 seconds.
- Storage & Visualization Layer — ThingSpeak stores each field as a time-series and renders live charts for all monitored parameters.

## 2.2 Data Flow

- HC-SR04 sensors trigger and measure pulse duration via dedicated GPIO pins on the ESP32-S3.
- Water level sensors deliver analog voltage signals read via ADC-capable GPIO pins.
- The microcontroller computes tank fill levels, cumulative usage, and time-to-empty estimates.
- Processed data is pushed to ThingSpeak across 8 fields every 15 seconds for cloud storage and visualization.

# 3. Component List

The following components are used in the Cirkit Designer simulation. All components are simulation-ready and were selected to match the hardware that would be deployed in a real petrochemical plant environment.

| Component | Quantity | Purpose |
|---|---|---|
| ESP32-S3 (Beta) | 1 | Microcontroller with built-in Wi-Fi for data processing and cloud upload |
| HC-SR04 Ultrasonic Sensor | 3 | Measures distance to water surface to compute tank fill level |
| Water Level Sensor | 3 | Provides analog signal as secondary confirmation of water level |
| 5V DC Power Supply | 1 | Powers the ultrasonic sensors (HC-SR04 requires 5V supply) |
| Breadboard | 1 | Organises wiring connections in simulation |
| Connecting Wires | As required | Links all components according to wiring diagram |

## 3.1 Component Descriptions

### ESP32-S3 Microcontroller

The ESP32-S3 is a high-performance, low-power microcontroller featuring a dual-core Xtensa LX7 processor, integrated Wi-Fi, and a rich set of GPIO pins including ADC-capable inputs. It serves as the central processing and communication unit of the system. The onboard Wi-Fi module enables the device to connect to a network and push sensor data to the ThingSpeak cloud platform without any external communication hardware.

### HC-SR04 Ultrasonic Sensor

The HC-SR04 is a widely used, cost-effective ultrasonic distance sensor. It emits a high-frequency sound pulse and measures the time taken for the echo to return. Using this time-of-flight measurement, the distance to the water surface is calculated. The fill level of each tank is then derived by subtracting this distance from the known tank height. The sensor operates at 5V and communicates via TRIG and ECHO digital pins.

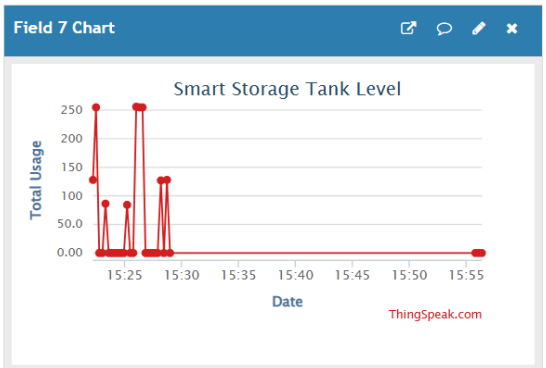
### Water Level Sensor

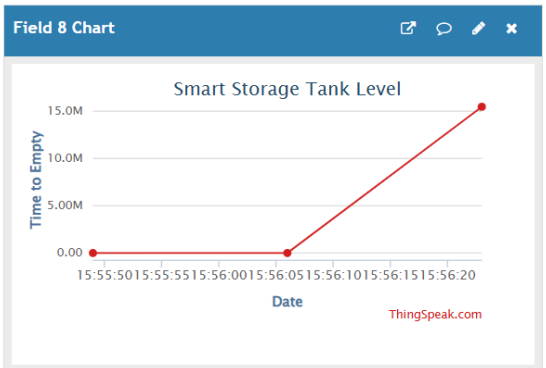
The water level sensor is a resistive-type analog sensor that provides a variable voltage output proportional to the depth of liquid contact. It is powered from the 3.3V pin of the ESP32-S3 and its signal output is connected to an ADC-capable GPIO pin. It serves as a secondary

measurement source to validate ultrasonic readings, particularly in scenarios where the ultrasonic sensor may be affected by surface turbulence or condensation.

### 5V DC Power Supply

A dedicated 5V DC power supply is used to power the HC-SR04 ultrasonic sensors. This is necessary because the HC-SR04 operates at 5V, while the ESP32-S3 outputs only 3.3V on its GPIO pins. Sharing a common ground between the power supply and the ESP32-S3 ensures signal integrity across all sensor connections.



# 4. Wiring Details

All wiring connections are defined based on the Cirkit Designer simulation schematic. The ESP32-S3 acts as the central hub to which all sensors are connected. The 5V DC power supply exclusively powers the HC-SR04 sensors, while the water level sensors are powered from the 3.3V output of the ESP32-S3. A shared GND bus connects all components.

## 4.1 ESP32-S3 Pin Assignments

| ESP32-S3 Pin | Connected To | Signal Type |
|---|---|---|
| GPIO 5 | TRIG — HC-SR04 Tank 1 | Digital Output |
| GPIO 17 | TRIG — HC-SR04 Tank 2 | Digital Output |
| GPIO 4 | TRIG — HC-SR04 Tank 3 | Digital Output |
| GPIO 18 | ECHO — HC-SR04 Tank 1 | Digital Input |
| GPIO 16 | ECHO — HC-SR04 Tank 2 | Digital Input |
| GPIO 2 | ECHO — HC-SR04 Tank 3 | Digital Input |
| GPIO 1 | SIG — Water Level Sensor 1 | Analog Input (ADC) |
| GPIO 3 | SIG — Water Level Sensor 2 | Analog Input (ADC) |
| GPIO 6 | SIG — Water Level Sensor 3 | Analog Input (ADC) |
| 3V3 | VCC — All Water Level Sensors | 3.3V Power |
| GND | GND — All Sensors and Power Supply | Common Ground |

## 4.2 HC-SR04 Ultrasonic Sensor Connections

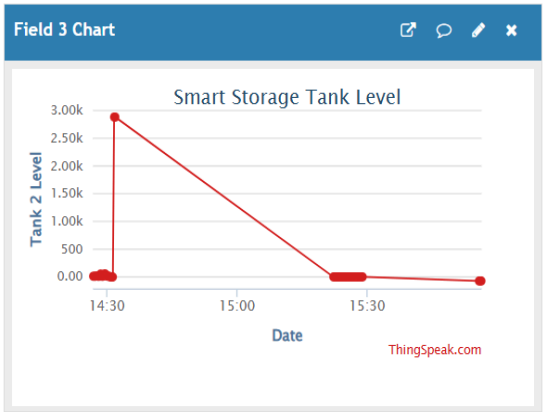
- VCC — Connected to the positive terminal of the 5V DC power supply.
- TRIG — Connected to respective ESP32-S3 GPIO output pins (GPIO 5, GPIO 17, GPIO 4).
- ECHO — Connected to respective ESP32-S3 GPIO input pins (GPIO 18, GPIO 16, GPIO 2).
- GND — Connected to the negative terminal of the 5V power supply and to the ESP32-S3 GND.

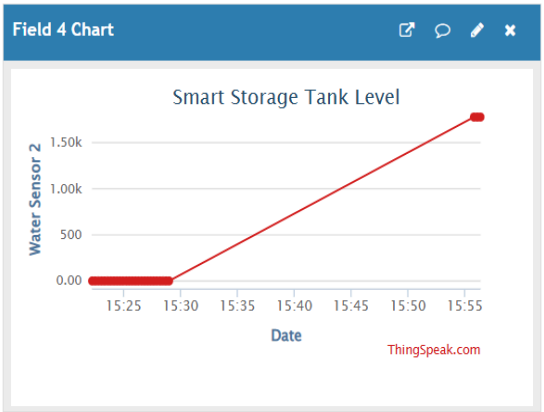
## 4.3 Water Level Sensor Connections

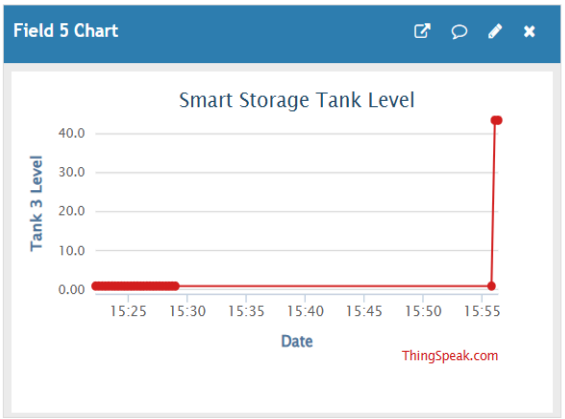
- SIG — Connected to respective ADC GPIO pins on the ESP32-S3 (GPIO 1, GPIO 3, GPIO 6).
- VCC — Connected to the 3V3 pin of the ESP32-S3.
- GND — Connected to the GND pin of the ESP32-S3.

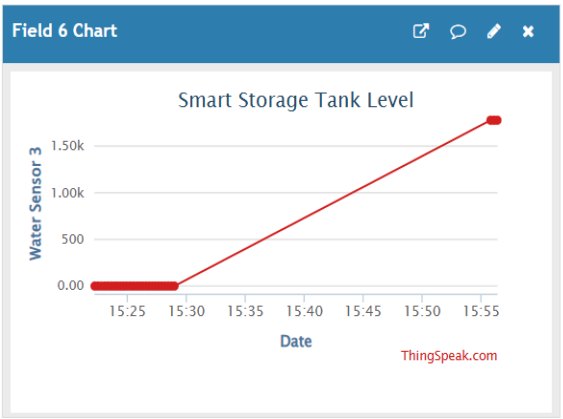
## 4.4 5V DC Power Supply Connections

- Positive terminal — Connected to VCC of all three HC-SR04 ultrasonic sensors.
- Negative terminal — Connected to GND of all sensors and to the GND pin of the ESP32-S3.



# 5. Code Documentation

The firmware is written in C++ for the Arduino framework and is uploaded to the ESP32-S3. It is responsible for initialising sensor GPIO pins, establishing a Wi-Fi connection, reading sensor values at regular intervals, computing tank metrics, and uploading results to ThingSpeak via HTTP GET requests.

## 5.1 Key Features

- Wi-Fi Connection — Connects to a defined SSID using the ESP32-S3 built-in Wi-Fi module. The system polls for connection status before proceeding.
- Sensor Initialisation — Configures TRIG pins as outputs and ECHO/SIG pins as inputs during the setup phase.
- Data Collection — Reads ultrasonic pulse duration and computes distance. Reads analog voltage from water level sensors via ADC.
- Data Processing — Calculates individual tank fill levels, cumulative total level, usage since the last reading cycle, and estimated time to empty.
- Data Upload — Sends all processed values to ThingSpeak across 8 fields using an HTTP GET request every 15 seconds.

## 5.2 ThingSpeak Field Mapping

| ThingSpeak Field | Variable | Description |
|---|---|---|
| Field 1 | levels[0] | Fill level of Tank 1 (cm) |
| Field 2 | waterValues[0] | Analog water sensor reading — Tank 1 |
| Field 3 | levels[1] | Fill level of Tank 2 (cm) |
| Field 4 | waterValues[1] | Analog water sensor reading — Tank 2 |
| Field 5 | levels[2] | Fill level of Tank 3 (cm) |
| Field 6 | waterValues[2] | Analog water sensor reading — Tank 3 |
| Field 7 | usage | Total liquid consumed since the previous cycle |
| Field 8 | timeToEmpty | Predicted time remaining until all tanks empty |

## 5.3 Complete Source Code

The following code is the complete firmware deployed on the ESP32-S3. No modifications should be made to this section without updating the corresponding ThingSpeak channel configuration.

```
#include <WiFi.h>
#include <HTTPClient.h>

// WiFi
#define WIFI_SSID "CirkitWifi"
#define WIFI_PASS ""

// ThingSpeak
#define API_KEY "Z3IK223CFZO23VJY"
#define SERVER  "http://api.thingspeak.com/update"

// Ultrasonic pins (3 tanks)
int trigPins[3] = {5, 17, 4};
int echoPins[3] = {18, 16, 2};

// Water level sensor pins
int waterPins[3] = {1, 3, 6};

// Tank data
float tankHeight = 100.0;
float levels[3];
int   waterValues[3];
float totalLevel = 0;
float prevTotalLevel = 0;
float usage = 0;
float timeToEmpty = 0;
unsigned long lastUploadTime = 0;

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) {
    pinMode(trigPins[i], OUTPUT);
    pinMode(echoPins[i], INPUT);
    pinMode(waterPins[i], INPUT);
  }
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");
}

void loop() {
  totalLevel = 0;
  for (int i = 0; i < 3; i++) {
    // Ultrasonic reading
    digitalWrite(trigPins[i], LOW);
    delayMicroseconds(2);
    digitalWrite(trigPins[i], HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPins[i], LOW);
    long duration = pulseIn(echoPins[i], HIGH);
    float distance = duration * 0.034 / 2;
    levels[i] = tankHeight - distance;

    // Water level sensor reading
    waterValues[i] = analogRead(waterPins[i]);
    totalLevel += levels[i];
    delay(200);
  }
```

```
  // Usage calculation
  if (prevTotalLevel != 0) usage = prevTotalLevel - totalLevel;
  else usage = 0;
  prevTotalLevel = totalLevel;

  // Time to empty
  timeToEmpty = (usage > 0) ? totalLevel / usage : 0;

  // Serial output
  Serial.println("------ Tank Data ------");
  for (int i = 0; i < 3; i++) {
    Serial.print("Tank "); Serial.print(i+1);
    Serial.print(" | Level: "); Serial.print(levels[i]);
    Serial.print(" | Water: "); Serial.println(waterValues[i]);
  }
  Serial.print("Total Usage: ");   Serial.println(usage);
  Serial.print("Time to Empty: "); Serial.println(timeToEmpty);

  // Upload to ThingSpeak every 15 seconds
  if (millis() - lastUploadTime >= 15000) {
    HTTPClient http;
    String url = String(SERVER) + "?api_key=" + API_KEY
      + "&field1=" + String(levels[0])
      + "&field2=" + String(waterValues[0])
      + "&field3=" + String(levels[1])
      + "&field4=" + String(waterValues[1])
      + "&field5=" + String(levels[2])
      + "&field6=" + String(waterValues[2])
      + "&field7=" + String(usage)
      + "&field8=" + String(timeToEmpty);
    http.begin(url);
    int response = http.GET();
    Serial.println(response > 0 ? "Upload Success" : "Upload Failed");
    http.end();
    lastUploadTime = millis();
  }
  delay(2000);
}
```

# 6. Circuit Schematic

The circuit schematic below was generated from the Cirkit Designer simulation environment. It illustrates the physical layout and wiring of the ESP32-S3, three HC-SR04 ultrasonic sensors, three water level sensors, and the 5V DC power supply. All connections match the pin assignments described in Section 4.

**URL:https://app.cirkitdesigner.com/project/52f8eb59-c9c1-48d0-9de5-3697dab5600b**

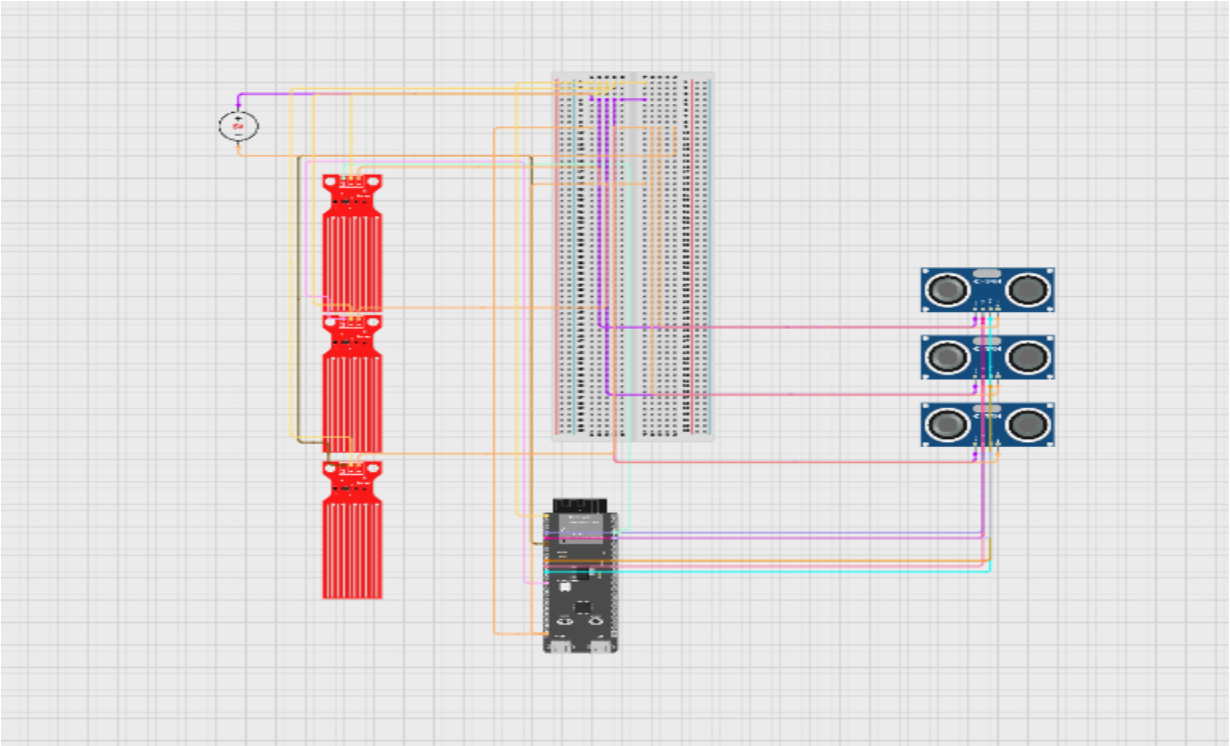


Figure 1: Cirkit Designer Simulation — ESP32-S3 with HC-SR04 and Water Level Sensors

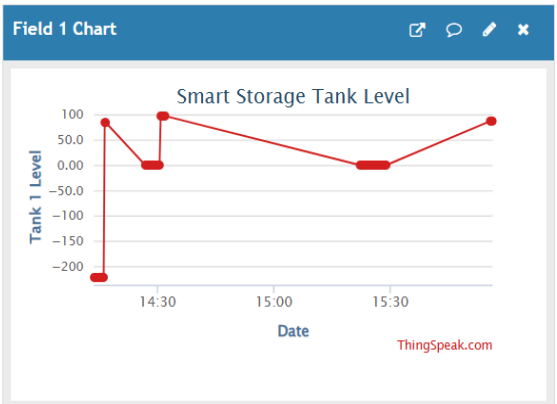
# 7. ThingSpeak Cloud Output

Upon successful execution of the firmware, the ESP32-S3 connects to the ThingSpeak cloud platform and begins uploading sensor data every 15 seconds. The ThingSpeak channel named 'Smart Storage Tank Level' visualizes all 8 fields as individual time-series charts, providing a real-time overview of tank conditions.

## 7.1 Observed Field Charts

- Field 1 — Tank 1 level (cm): Displays the ultrasonic-derived fill level of Tank 1 over time.
- Field 2 — Water Sensor 1: Analog ADC reading from the water level sensor in Tank 1.
- Field 3 — Tank 2 level (cm): Fill level of Tank 2 measured by the second HC-SR04 sensor.
- Field 4 — Water Sensor 2: ADC output from the water level sensor in Tank 2.
- Field 5 — Tank 3 level (cm): Fill level of Tank 3 as measured by the third HC-SR04 sensor.
- Field 6 — Water Sensor 3: Analog reading from the water level sensor in Tank 3.
- Field 7 — Total Usage: Cumulative consumption calculated as the difference between consecutive total level readings.
- Field 8 — Time to Empty: Predicted time remaining before total depletion, based on the current rate of usage.
-

**URL:https://thingspeak.mathworks.com/channels/3327870**



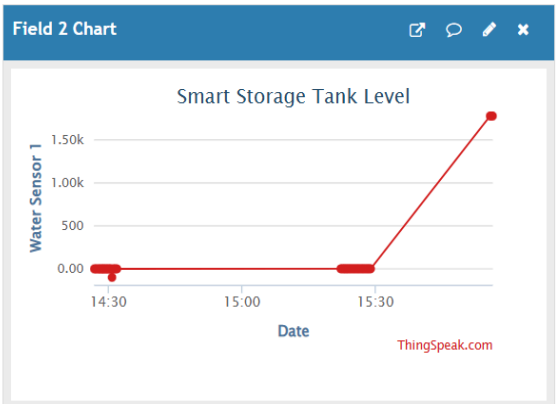




Figure 2: ThingSpeak Channel — Live Field Charts

## 8. Project Deliverables & Milestones

The complete project is structured across five milestones, each building upon the previous to deliver a fully integrated IoT and machine learning system for petrochemical tank monitoring. The current documentation covers Milestone 1 in full.

| Milestone | Title | Key Deliverables | Status |
|---|---|---|---|
| 1 | System Architecture Design | Circuit design in Cirkit Designer, pin mapping, ThingSpeak integration, firmware | Complete |
| 2 | Dataset Generation & ML Model Development | Synthetic tank-level dataset, trained ML model for prediction and risk classification | Upcoming |
| 3 | Live Data Simulation & Model Integration | Real-time sensor simulation, cloud upload, ML prediction on live data | Upcoming |
| 4 | Analytics Output & Dashboard Visualization | Structured prediction output (CSV), Power BI dashboard with trends and alerts | Upcoming |
| 5 | Result Analysis & Industry Interpretation | Final analysis report, safety improvement demonstration, inventory management insights | Upcoming |

## 9. Conclusion

Milestone 1 establishes the foundational hardware and cloud connectivity layer for the Smart Storage Tank Level and Usage Prediction System. The Cirkit Designer simulation demonstrates a functional circuit comprising an ESP32-S3 microcontroller, three HC-SR04 ultrasonic sensors, three water level sensors, and a 5V DC power supply. The firmware successfully reads sensor data, computes tank metrics, and transmits results to the ThingSpeak cloud platform at 15-second intervals.

The live ThingSpeak charts confirm that all 8 fields — including individual tank levels, analog water sensor readings, total usage, and predicted time-to-empty — are being captured and visualized correctly. This milestone provides the data pipeline required for the machine learning and dashboard components that will be developed in subsequent milestones.